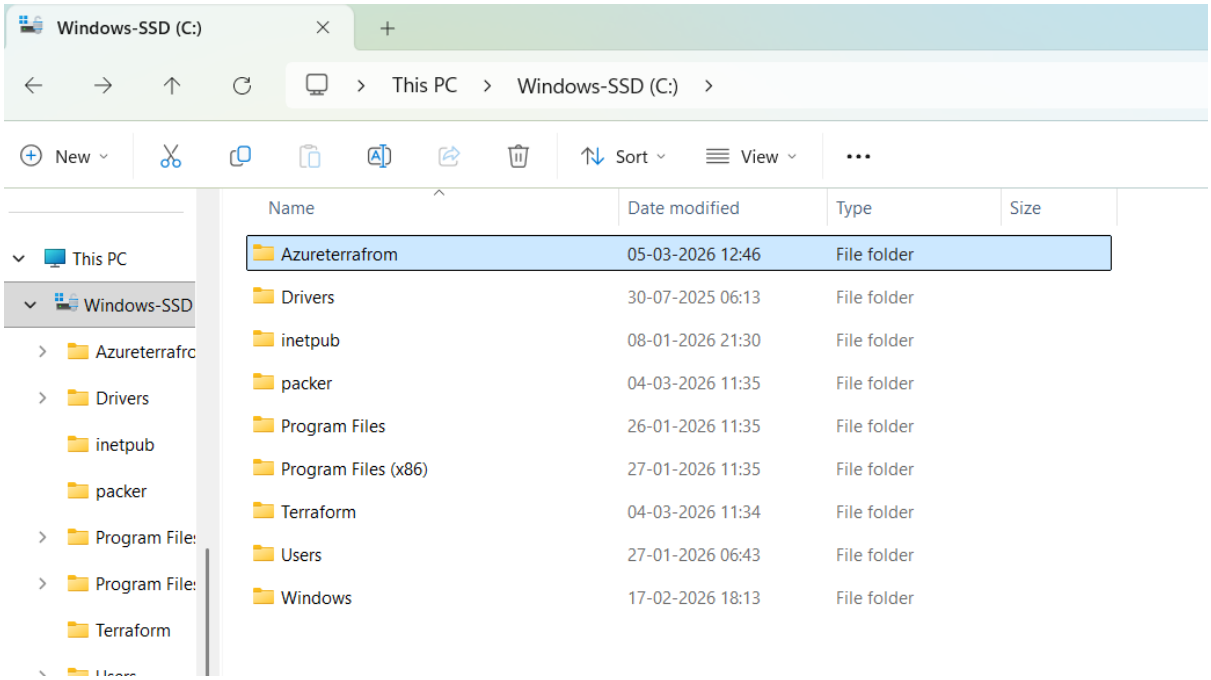
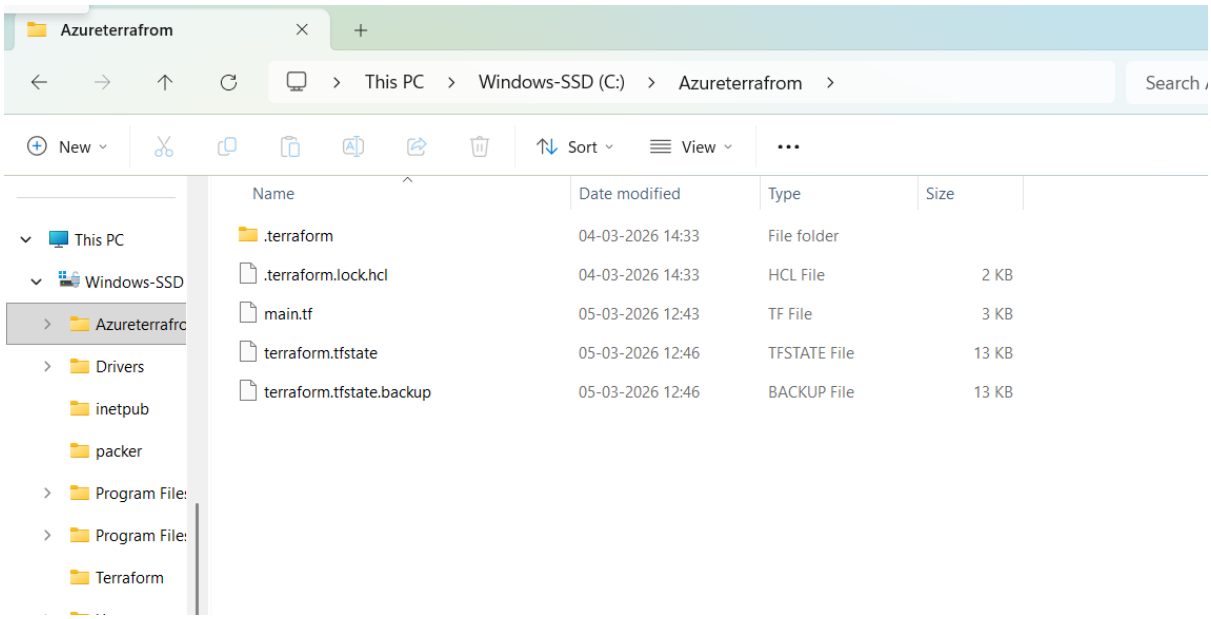
Peering Using Terraform

- Create a New Folder -- **Azureterraform**

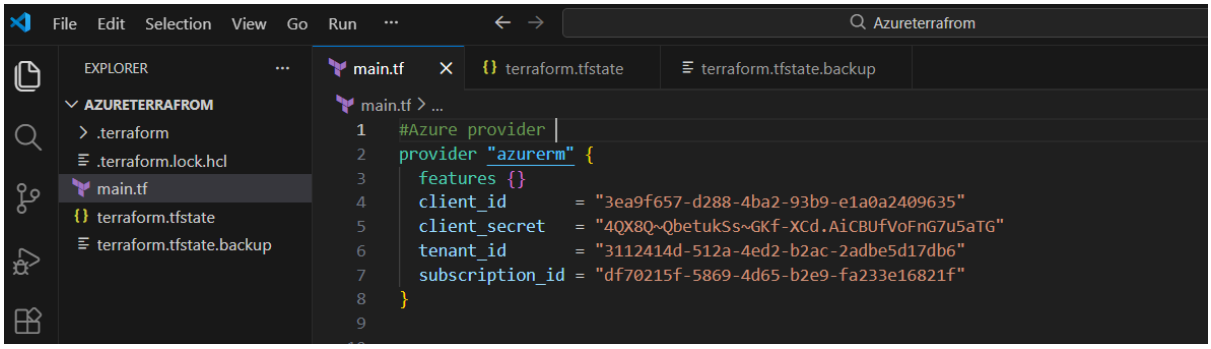


- Select -- **Azureterraform**



- We Have all the files of terraform
- Right Click and open the terminal

- In terminal enter **code .**
- It redirect to the visual studio code



- We Have Providers and Resources
- **features{ }** – For Authorization And Authentication

- Create Resource Group

```
#Azure Resource Group creation
resource "azurerm_resource_group" "nandarg" {
  name     = "Nanda-RG"
  location = "westeurope"
}
```

- Create Virtual Network and Subnet

```
#Azure Virtual Network Creation
resource "azurerm_virtual_network" "nandavnet1" {
  name                = "Nanda-Vnet1"
  location            = "westeurope"
  resource_group_name = "Nanda-RG"
  address_space       = ["10.0.0.0/16"]
}

#Azure Subnet Creation
resource "azurerm_subnet" "nandasubnet1" {
  name                 = "Nsubnet01"
  resource_group_name  = "Nanda-RG"
  virtual_network_name = "Nanda-Vnet1"
  address_prefixes     = ["10.0.1.0/24"]
  depends_on = [ azurerm_virtual_network.nandavnet1 ]
}
```

➢ Create another Virtual Network and Subnet

```
#Azure Virtual Network 2 Creation
resource "azurerm_virtual_network" "nandavnet2" {
  name                = "Nanda-Vnet2"
  location            = "westeurope"
  resource_group_name = "Nanda-RG"
  address_space       = ["10.1.0.0/16"]
}

#Azure Subnet 2 Creation
resource "azurerm_subnet" "nandasubnet2" {
  name                 = "Nsubnet02"
  resource_group_name  = "Nanda-RG"
  virtual_network_name = "Nanda-Vnet2"
  address_prefixes     = ["10.1.1.0/24"]
  depends_on = [ azurerm_virtual_network.nandavnet2 ]
}
```

- Subnets may create first instead of Virtual Network. We get error, to avoid these type of errors we use depends_on = [azurerm_virtual_network.vnetname]
- It will execute vnet first and then subnet.

```
depends_on = [ azurerm_virtual_network.nandavnet2 ]
```

➢ We Have to do Peering for Both the Virtual Networks.

➢ Eg: peer1-to-peer2

peer2-to-peer1

✓ Peering

```
#peering peer1-to-peer2
resource "azurerm_virtual_network_peering" "peer1" {
  name                      = "peer1-to-peer2"
  resource_group_name       = "Nanda-RG"
  virtual_network_name      = azurerm_virtual_network.nandavnet2.name
  remote_virtual_network_id = azurerm_virtual_network.nandavnet1.id
}

resource "azurerm_virtual_network_peering" "peer2" {
  name                      = "peer2-to-peer1"
  resource_group_name       = "Nanda-RG"
  virtual_network_name      = azurerm_virtual_network.nandavnet1.name
  remote_virtual_network_id = azurerm_virtual_network.nandavnet2.id
}
```

➢ Go to **powershell terminal**

- Enter **terraform init** -- to install the terraform plugins

```
PS C:\Azureterrafrom> code .
PS C:\Azureterrafrom> terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/azurerm v4.62.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Azureterrafrom> terraform validate
Success! The configuration is valid.
```

➢ Next **terraform validate**

For Configuration

- Next **terraform plan**

  It will show what will be executed.

```
PS C:\Azureterrafrom> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_resource_group.nandarg will be created
  + resource "azurerm_resource_group" "nandarg" {
      + id       = (known after apply)
      + location = "westeurope"
      + name     = "Nanda-RG"
    }

  # azurerm_subnet.nandasubnet1 will be created
  + resource "azurerm_subnet" "nandasubnet1" {
      + address_prefixes                            = [
          + "10.0.1.0/24",
        ]
      + default_outbound_access_enabled             = true
      + id                                          = (known after apply)
      + name                                        = "Nsubnet01"
      + private_endpoint_network_policies           = "Disabled"
      + private_link_service_network_policies_enabled = true
      + resource_group_name                         = "Nanda-RG"
      + virtual_network_name                        = "Nanda-Vnet1"
    }

  # azurerm_virtual_network.nandavnet1 will be created
  + resource "azurerm_virtual_network" "nandavnet1" {
      + address_space                       = [
          + "10.0.0.0/16",
        ]
      + dns_servers                         = (known after apply)
      + guid                                = (known after apply)
      + id                                  = (known after apply)
      + location                            = "westeurope"
      + name                                = "Nanda-Vnet1"
      + private_endpoint_vnet_policies      = "Disabled"
      + resource_group_name                 = "Nanda-RG"
      + subnet                              = (known after apply)
    }

Plan: 3 to add, 0 to change, 0 to destroy.
```

- Next terraform **apply –auto-approve**

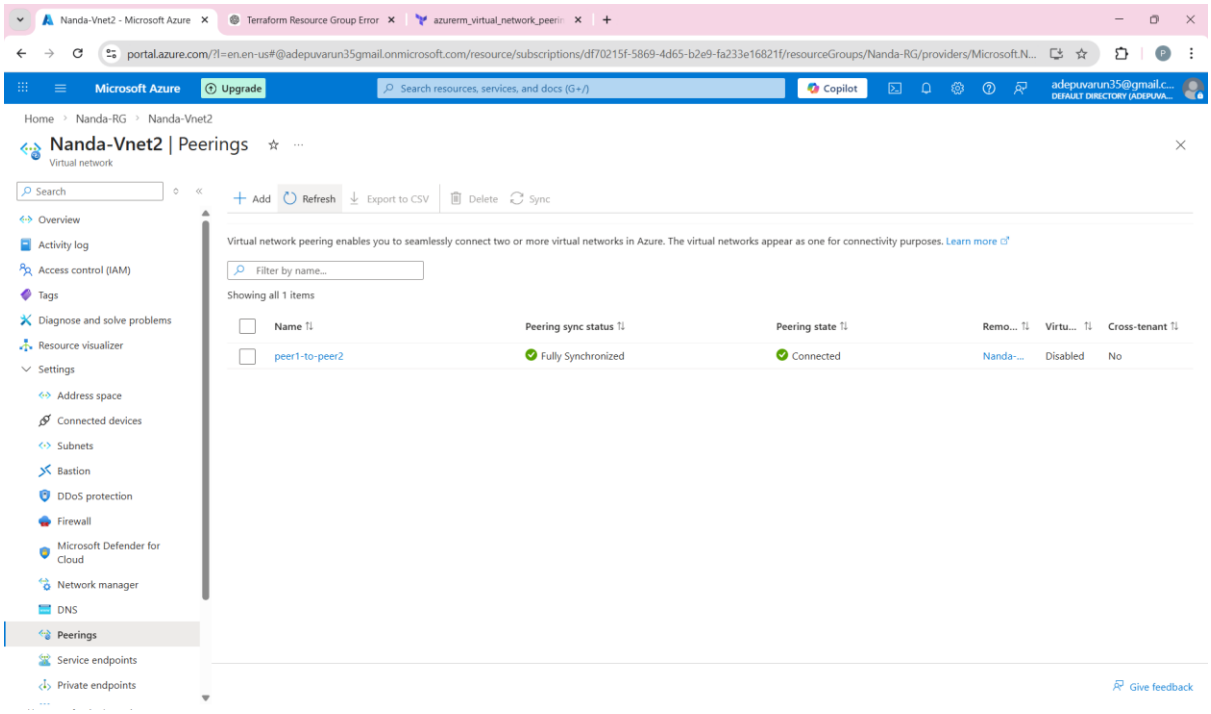
```
PS C:\Azureterrafrom> terraform apply --auto-approve
azurerm_resource_group.nandarg: Refreshing state... [id=/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourceGroups/Nanda-RG]
azurerm_virtual_network.nandavnet1: Refreshing state... [id=/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourceGroups/Nanda-RG/providers/Microsoft.
Network/virtualNetworks/Nanda-Vnet1]
azurerm_subnet.nandasubnet1: Refreshing state... [id=/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourceGroups/Nanda-RG/providers/Microsoft.Network
/virtualNetworks/Nanda-Vnet1/subnets/Nsubnet01]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_subnet.nandasubnet2 will be created
  + resource "azurerm_subnet" "nandasubnet2" {
      + address_prefixes                            = [
          + "10.1.1.0/24",
        ]
      + default_outbound_access_enabled             = true
      + id                                          = (known after apply)
      + name                                        = "Nsubnet02"
      + private_endpoint_network_policies           = "Disabled"
      + private_link_service_network_policies_enabled = true
      + resource_group_name                         = "Nanda-RG"
      + virtual_network_name                        = "Nanda-Vnet2"
    }

  # azurerm_virtual_network.nandavnet2 will be created
  + resource "azurerm_virtual_network" "nandavnet2" {
      + address_space                       = [
          + "10.1.0.0/16",
        ]
      + dns_servers                         = (known after apply)
      + guid                                = (known after apply)
      + id                                  = (known after apply)
      + location                            = "westeurope"
      + name                                = "Nanda-Vnet2"
      + private_endpoint_vnet_policies      = "Disabled"
      + resource_group_name                 = "Nanda-RG"
      + subnet                              = (known after apply)
    }
```

✓ **Terraform apply** will create the required Virtual machines, Virtuals machines.

```
Plan: 2 to add, 0 to change, 0 to destroy.
azurerm_virtual_network.nandavnet2: Creating...
azurerm_virtual_network.nandavnet2: Creation complete after 10s [i
crosoft.Network/virtualNetworks/Nanda-Vnet2]
azurerm_subnet.nandasubnet2: Creating...
azurerm_subnet.nandasubnet2: Still creating... [00m10s elapsed]
azurerm_subnet.nandasubnet2: Creation complete after 12s [id=/subs
.Network/virtualNetworks/Nanda-Vnet2/subnets/Nsubnet02]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

➢ Next **terraform apply** –auto-approve -- for peering

```
PS C:\Azureterrafrom> terraform apply --auto-approve
azurerm_resource_group.nandarg: Refreshing state... [id=/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourceGro
azurerm_virtual_network.nandavnet1: Refreshing state... [id=/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourc
Network/virtualNetworks/Nanda-Vnet1]
azurerm_virtual_network.nandavnet2: Refreshing state... [id=/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourc
Network/virtualNetworks/Nanda-Vnet2]
azurerm_subnet.nandasubnet2: Refreshing state... [id=/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourceGroups
/virtualNetworks/Nanda-Vnet2/subnets/Nsubnet02]
azurerm_subnet.nandasubnet1: Refreshing state... [id=/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourceGroups
/virtualNetworks/Nanda-Vnet1/subnets/Nsubnet01]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_virtual_network_peering.peer1 will be created
  + resource "azurerm_virtual_network_peering" "peer1" {
      + allow_forwarded_traffic                = false
      + allow_gateway_transit                  = false
      + allow_virtual_network_access           = true
      + id                                     = (known after apply)
      + name                                   = "peer1-to-peer2"
      + peer_complete_virtual_networks_enabled = true
      + remote_virtual_network_id              = "/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourceGroups/Na
rtualNetworks/Nanda-Vnet1"
      + resource_group_name                    = "Nanda-RG"
      + use_remote_gateways                    = false
      + virtual_network_name                   = "Nanda-Vnet2"
    }

  # azurerm_virtual_network_peering.peer2 will be created
  + resource "azurerm_virtual_network_peering" "peer2" {
      + allow_forwarded_traffic                = false
      + allow_gateway_transit                  = false
      + allow_virtual_network_access           = true
      + id                                     = (known after apply)
      + name                                   = "peer2-to-peer1"
      + peer_complete_virtual_networks_enabled = true
      + remote_virtual_network_id              = "/subscriptions/df70215f-5869-4d65-b2e9-fa233e16821f/resourceGroups/Na
rtualNetworks/Nanda-Vnet2"
      + resource_group_name                    = "Nanda-RG"
      + use_remote_gateways                    = false
      + virtual_network_name                   = "Nanda-Vnet1"
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
azurerm_virtual_network_peering.peer1: Creating...
azurerm_virtual_network_peering.peer2: Creating...
azurerm_virtual_network_peering.peer1: Still creating... [00m10s elapsed]
azurerm_virtual_network_peering.peer2: Still creating... [00m10s elapsed]
azurerm_virtual_network_peering.peer2: Still creating... [00m20s elapsed]
azurerm_virtual_network_peering.peer1: Still creating... [00m20s elapsed]
azurerm_virtual_network_peering.peer1: Still creating... [00m30s elapsed]
azurerm_virtual_network_peering.peer2: Still creating... [00m30s elapsed]
azurerm_virtual_network_peering.peer1: Creation complete after 31s [id=/su
/Microsoft.Network/virtualNetworks/Nanda-Vnet2/virtualNetworkPeerings/peer
azurerm_virtual_network_peering.peer2: Still creating... [00m40s elapsed]
azurerm_virtual_network_peering.peer2: Still creating... [00m50s elapsed]
azurerm_virtual_network_peering.peer2: Still creating... [01m00s elapsed]
azurerm_virtual_network_peering.peer2: Creation complete after 1m2s [id=/s
s/Microsoft.Network/virtualNetworks/Nanda-Vnet1/virtualNetworkPeerings/pee

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

- Peering Done **Succesfully**